

Audit sécurité applicative & conformité RGPD

RSSI / DPO — Responsable Sécurité & RGPD

Projet	CleanUx / brio — marketplace multi-métiers (Laravel 12 + apps mobiles Expo)
Destinataire	RSSI / DPO — Responsable Sécurité & RGPD
Date d'audit	24 juin 2026
Référence	Réactualisation de l'audit plateforme du 8 juin 2026
Confidentialité	Document interne — confidentiel

Légende sévérité : ● Critique · ● Élevé · ● Moyen · ● Faible · ● Sain / Corrigé

1. Résumé exécutif

L'évolution depuis l'audit du 8 juin est **spectaculaire** : la quasi-totalité des findings sécurité et RGPD ont été **remédiés et, pour beaucoup, couverts par des tests** de non-régression. Le socle est aujourd'hui mature :

- **Secrets** : aucun `.env` réel committé, valeurs factices uniquement, aucun secret en dur dans le code.
- **Authentification** : 2FA admin imposée par défaut, rate-limiting login (5/min), expiration des tokens Sanctum.
- **Autorisation** : l'IDOR du module Qualité est corrigé (policies + `authorize()`), gating de rôle `role:employee` sur les routes provider.
- **RGPD** : droit à l'oubli **fonctionnel et testé**, anonymisation étendue, export, rétention/purge automatisée, chiffrement des PII KYC/KYB, masquage des PII dans logs et audit trail.
- **Réseau** : headers de sécurité (HSTS, CSP, nosniff...), CORS sans wildcard, signature + idempotence sur tous les webhooks entrants.

Écarts résiduels : tous de niveau **Moyen ou Faible** et relevant de la configuration/dépendances, pas d'une refonte — dépendances vulnérables (l'affirmation « 0 CVE » du README est fausse), consentement non persisté côté serveur, politique de mot de passe au défaut Laravel, `TRUSTED_PROXIES=*` dans l'exemple de prod.

Avis conformité RGPD : GO conditionnel. Aucun bloqueur critique. Quatre correctifs de configuration/dépendances suffisent à sécuriser le go-live.

2. Gestion des secrets

2.1 ● `.env` non committé, secrets factices, aucun hardcode

`git ls-files` confirme qu'aucun `.env*` réel n'est suivi (`.gitignore:32`). Le `.env` local ne contient que des placeholders (`STRIPE_SECRET=sk_test_xxx` ...), clés IA/AWS vides. Aucun secret en dur dans `app/` / `config/` (tous via `env()`).

2.2 ● Hygiène du `.env` local

`APP_DEBUG=true`, `DB_PASSWORD=root` (local), et une **PII réelle** (`OPS_MONITORING_NOTIFY_EMAIL=mm.darouch@hotmail.com`). Sans impact prod (`.env.production.example` met `APP_DEBUG=false`), mais à nettoyer du fichier partagé en équipe.

3. Authentification & autorisation

3.1 ● Politique de mot de passe = défaut Laravel (8 caractères, sans complexité)

`PasswordValidationRules.php:17` utilise `Password::default()` sans `Password::defaults(...)` configuré → min. 8 caractères, sans exigence de casse/chiffres ni vérification de compromission, sur une plateforme de paiement.

- **Remédiation** : `Password::defaults(fn () => Password::min(12)→mixedCase()→numbers()→uncompromised())` dans `AppServiceProvider::boot()`.

3.2 ● 2FA, rate-limiting, autorisation — sains

- 2FA admin **ON par défaut** (`config/auth.php:129`, middleware `Enforce2FA`).
- Rate-limit login 5/min par email+IP (`FortifyServiceProvider.php:38-42`).
- IDOR Qualité (C1) **corrigé** : `MissionQualityInspectionPolicy` + `authorize()` côté client et provider. Achat d'assurance (M2) protégé par contrôle d'ownership.
- Gating de rôle `role:employe` sur la surface provider (`routes/api/provider.php:33`).

3.3 ● Groupe `/payouts` (lecture seule) sans `role:employe`

`routes/api/provider.php:167` : groupe read-only gardé par `auth:sanctum` seul. À uniformiser, impact faible.

4. Réseau — headers & CORS

4.1 ● `TRUSTED_PROXIES=*` dans l'exemple de production

`.env.production.example:115`. Si exposé sans LB filtrant, permet le spoofing de `X-Forwarded-For` → contournement du rate-limiting par IP et falsification des IP dans l'audit trail.

- **Remédiation** : restreindre à la plage CIDR du load-balancer.

4.2 ● Headers & CORS — sains

`SecurityHeaders.php` : HSTS (prod+HTTPS), `X-Content-Type-Options: nosniff`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, **CSP stricte**, `Cache-Control: no-store` sur réponses authentifiées. `config/cors.php` sans wildcard. `ForceHttps` + cookies sécurisés en prod.

5. Conformité RGPD

5.1 ● Droit à l'oubli — corrigé et testé

`Console/Kernel.php:35` planifie désormais `gdpr:execute-erasures` (nom aligné avec la commande). Un **test** (`ScheduleIntegrityTest`) asserte que toute commande planifiée est résolvable. L'article 17 est de nouveau exécutable automatiquement.

5.2 ● Anonymisation étendue

`DataErasureService::anonymizeCorePii()` couvre désormais `bookings` (adresse/ville/CP/notes), `feedback`, `complaint_cases`, `notifications.data`, `analytics_events` (`user_id` → null), `audit_events` (labels).
Conservation comptable des montants/dates délibérée (obligation légale).

5.3 ● Consentement non persisté côté serveur (art. 7.1)

Le bandeau cookies stocke le choix **uniquement en** `localStorage` /cookie navigateur (`cookie-banner.blade.php:87-101`). Aucune table de consentement serveur, aucun consentement enregistré à l'inscription.

- **Risque** : impossibilité de **prouver** le consentement et son horodatage (charge de la preuve art. 7.1) face à un contrôle CNIL/APD.
- **Remédiation** : journaliser chaque choix côté serveur (`user_id`/IP hashée, version, scope, timestamp).

5.4 ● Rétention, export & PII — sains

Purge des exports PII expirés, rétention analytics 425 j, masquage des emails dans `audit_events` et hash des PII dans les logs marketing, chiffrement des données KYC/KYB sensibles (`EncryptedArrayFallback` / `EncryptedStringFallback`), photos sur disque **privé** hors webroot. Pages légales présentes (privacy, terms, mentions, cookies).

6. ● Dépendances vulnérables — « 0 CVE » du README est faux

`composer audit` (24/06/2026) → **9 advisories Medium / 3 paquets** :

- `guzzlehttp/guzzle` < 7.12.1 (CVE-2026-55767, CVE-2026-55568)
- `guzzlehttp/psr7` < 2.12.1 (3 CVE CRLF / host confusion)
- `web-token/jwt-library` — **algorithm confusion** (GHSA-jc38-x7x8-2xc8) + 2 autres

`npm audit` racine = 0 vulnérabilité.

- **Remédiation** : `composer update guzzlehttp/guzzle guzzlehttp/psr7 web-token/jwt-library` ; rendre `composer audit` bloquant en CI ; corriger l'affirmation du README.

7. Webhooks & monitoring — sains

● Signature vérifiée + idempotence partout : Stripe/Connect (`constructEvent`, `firstOrCreate` sur `event_id`, anti-replay), Onfido (HMAC-SHA256 `hash_equals`), Twilio (HMAC-SHA1). Fallback d'idempotence déterministe (hash du payload) au lieu d'aléatoire. Sentry avec `send_default_pii=false`. Kill-switches feature-flags opérationnels (lecture DB au runtime).

8. Synthèse des écarts résiduels

Sévérité	Écart	Preuve
● Moyen	Dépendances vulnérables (dont algorithm confusion JWT)	<code>composer audit</code> 24/06
● Moyen	Consentement non persisté serveur (art. 7.1)	<code>cookie-banner.blade.php:87-101</code>
● Moyen	Politique de mot de passe = défaut (8 car.)	<code>PasswordValidationRules.php:17</code>

Sévérité	Écart	Preuve
● Moyen	<code>TRUSTED_PROXIES=*</code> exemple prod	<code>.env.production.example:115</code>
● Faible	PII perso + <code>APP_DEBUG=true</code> dans <code>.env</code> local	<code>.env:5,99</code>
● Faible	Groupe <code>/payouts</code> sans <code>role:employee</code> (read-only)	<code>routes/api/provider.php:167</code>

9. Avis global & conditions de go-live

Conformité RGPD : GO conditionnel. Le socle est mature et fonctionnel (droit à l'oubli testé, anonymisation complète, chiffrement PII, masquage logs/audit, bandeau conforme). **Aucun bloqueur critique.**

4 conditions avant go-live (toutes Moyennes, effort faible) :

1. Mettre à jour les 3 dépendances vulnérables + corriger le README.
2. Persister le consentement côté serveur (charge de la preuve).
3. Durcir la politique de mot de passe.
4. Restreindre `TRUSTED_PROXIES` à la plage du LB.

Réserve : `composer audit` a été exécuté ; les tests de sécurité automatisés n'ont pas été rejoués (config vérifiée, pas de runtime).